บทที่ 2

การจัดเลย์เอาต์หน้าจอด้วย Flexbox

ระบบเลย์เอาต์ (Layout System) ของ React Native ทำงานโดยการอ้างอิงและประยุกต์ใช้ข้อกำหนดของ **Flexbox** เป็นหลัก (React Native, 2024) ซึ่งมีแนวคิดคล้ายกับ CSS บนเว็บ แต่มีคุณสมบัติเฉพาะตัวบางประการที่ออกแบบมาเพื่อให้แสดงผลบนขนาดหน้าจอสมาร์ทโฟนที่มีสัดส่วนหลากหลายได้อย่างสม่ำเสมอ

2.1 การเขียนและประกาศสไตล์ใน React Native

ใน React Native เราไม่มีคลาสสไตล์ชีทแบบ CSS ไฟล์แยกภายนอก (เช่น `.css`) แต่จะใช้งานผ่านออบเจกต์ JavaScript ในรูปแบบ **CSS-in-JS** ซึ่งสามารถเขียนประกาศได้ 2 รูปแบบหลัก:

1. แบบ Inline Styles

เป็นการส่งออบเจกต์สไตล์เข้าไปยังพร็อพส์ `style` ของคอมโพเนนต์โดยตรง เหมาะสำหรับใช้กำหนดคุณสมบัติที่เปลี่ยนแปลงตามสถานะตัวแปร หรือทดสอบเลย์เอาต์แบบเร่งด่วน:

```
<View style={{ width: 100, height: 100, backgroundColor: 'red' }} />
```

2. แบบ StyleSheet.create (แนะนำ)

เป็นการประกาศชุดสไตล์ภายนอกคอมโพเนนต์ (React Native, 2024) ทำให้แยกส่วนการแสดงผลและสไตล์ชัดเจน ตัวคอมไพเลอร์ของ React Native สามารถแปลรหัสและแคชโครงสร้างสไตล์นี้ส่งไปยังฝั่งระบบปฏิบัติการได้มีประสิทธิภาพสูงกว่า:

```
import { StyleSheet, View } from 'react-native';

export default function App() {
  return <View style={styles.box} />;
}

const styles = StyleSheet.create({
  box: {
    width: 100,
    height: 100,
    backgroundColor: '#38bdf8',
  },
});
```

2.2 โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต (Box Model)

การกำหนดกล่องสไตล์ชีทประกอบด้วยคุณสมบัติสำคัญที่เป็นพื้นฐานในการจำกัดพื้นที่ ได้แก่:

• `width` & `height`: กำหนดความกว้างและความสูง สามารถใส่เป็นค่าพิกเซลเดี่ยว (เช่น `150`) หรือค่าเปอร์เซ็นต์ (เช่น `'50%'`)

• `margin` **(ระยะขอบภายนอก):** ระยะห่างระหว่างขอบกล่องภายนอกไปยังกล่องข้างเคียง

• `padding` **(ระยะขอบภายใน):** ระยะห่างระหว่างขอบกล่องด้านในไปยังวัตถุหรือเนื้อหาข้างในกล่อง

• `border` **(เส้นขอบ):** กำหนดขนาดและสีขอบด้วย `borderWidth`, `borderColor`

• `borderRadius` **(มุมโค้งมน):** ใช้ตัดมุมกล่องให้โค้งงอ (หากต้องการตัดกล่องให้กลมสนิท ให้กำหนด `borderRadius` เป็นครึ่งหนึ่งของขนาดความกว้าง/สูง)

```
// ตัวอย่างกล่องโปรไฟล์ที่ใช้คุณสมบัติสเปซขอบ
const styles = StyleSheet.create({
  card: {
    width: 300,
    padding: 16,          // ระยะเว้นข้อความข้างใน
    margin: 8,            // ระยะเว้นนอกการ์ด
    borderWidth: 2,
    borderColor: '#e2e8f0',
    borderRadius: 12,    // ทำมุมการ์ดโค้งมนสไตล์การ์ดไอที
  }
});
```

2.3 ทิศทางการวางแกนด้วย flexDirection และ flex

หัวใจของการจัดตำแหน่งใน React Native คือทิศทางการวิ่งของข้อมูลในกลุ่มคอนเทนเนอร์ (Container):

1. คุณสมบัติ flexDirection (ทิศทางแกนหลัก)

กำหนดทิศทางเรียงตัวของคอมโพเนนต์ลูก (Children) ทั้งหมดภายในกล่องคอนเทนเนอร์นั้น:

ตารางที่ 2.1 ค่าคงที่และทิศทางการจัดวางของ flexDirection

ค่าคงที่	คำอธิบาย
column (ค่าเริ่มต้น)	เรียงตัวจาก บนลงล่าง (แกนหลักวิ่งแนวตั้ง,

	แกนรองวิ่งแนวนอน)
row	เรียงตัวจาก ซ้ายไปขวา (แกนหลักวิ่งแนวนอน, แกนรองวิ่งแนวตั้ง)
column-reverse	เรียงตัวจาก ล่างขึ้นบน ย้อนกลับตามแนวตั้ง
row-reverse	เรียงตัวจาก ขวาไปซ้าย ย้อนกลับตามแนวนอน

ข้อควรจำ (Important): ความแตกต่างสำคัญ: บนเว็บบราวเซอร์ ค่าเริ่มต้นของ Flexbox จะเป็น `row` (แนวนอน) แต่ใน **React Native** ค่าเริ่มต้นถูกตั้งใจตั้งไว้เป็น `column` (แนวตั้ง) เพื่อสอดคล้องกับพฤติกรรมการเลื่อนหน้าจอสมาร์ทโฟน

2. คุณสมบัติ flex (สัดส่วนการยืดขยาย)

การสเกลพื้นที่ของแต่ละกล่อง หากกำหนด `flex: 1` หมายความว่าให้กล่องนั้นครอบครองพื้นที่ว่างทั้งหมดที่เหลืออยู่ในแถวนั้น

```
// กล่องคอนเทนเนอร์ขนาดใหญ่คลุมหน้าจอ และแบ่งพื้นที่ออกเป็นสองบล็อกเท่ากัน
const styles = StyleSheet.create({
  container: {
    flex: 1, // ขยายครอบพื้นที่หน้าจอทั้งหมด
    flexDirection: 'row', // จัดวางแบบแนวนอน (ซ้ายไปขวา)
  },
  leftColumn: {
    flex: 1, // เอาพื้นที่ครึ่งหนึ่งทางซ้าย
    backgroundColor: '#1e293b',
  },
  rightColumn: {
    flex: 1, // เอาพื้นที่อีกครึ่งหนึ่งทางขวา
    backgroundColor: '#3b82f6',
  }
});
```

2.4 การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems

เมื่อกำหนดทิศทางแกนแล้ว เราจะใช้คำสั่งอีก 2 ชุดในการระบุการกระจายตัวของวัตถุ:

1. justifyContent (จัดวางในทิศทางแกนหลัก)

- `flex-start`: ชิดต้นสายของแกนหลัก
- `center`: จัดวัตถุทั้งหมดให้อยู่ตรงกึ่งกลางของแกนหลัก
- `flex-end`: ชิดท้ายสายของแกนหลัก

• `space-between`: กระจายตัวบล็อกโดยเว้นระยะห่างตรงกลางให้กว้างที่สุดเท่ากัน (หัวท้ายติดขอบ)

• `space-around`: กระจายตัวโดยเว้นระยะรอบข้างบล็อกเท่า ๆ กัน (หัวท้ายมีช่องว่างครึ่งหนึ่งของตรงกลาง)

• `space-evenly`: กระจายช่องว่างระหว่างกล่องทุกบล็อกเท่ากันทั้งหมด รวมถึงหัวและท้าย

2. alignItems (จัดวางในทิศทางแกนรอง)

• `stretch` (ค่าเริ่มต้น): ยืดขยายบล็อกให้ยาวจนเต็มขอบแกนรอง (จะใช้งานได้เมื่อไม่มีการฟิกขนาดด้านนั้น)

• `flex-start`: ชิดจุดเริ่มต้นแกนรอง

• `center`: จัดวัตถุลูกทั้งหมดให้กึ่งกลางแนวแกนรอง

• `flex-end`: ชิดจุดสิ้นสุดแกนรอง

2.5 การขึ้นบรรทัดใหม่และการเว้นช่องว่าง (flexWrap & gap)

1. คุณสมบัติ flexWrap

ในกรณีที่วัตถุลูกมีความกว้างรวมกันเกินความกว้างคอนเทนเนอร์ หากตั้งค่า `flexWrap: 'nowrap'` (ค่าเริ่มต้น) กล่องจะล้นออกไปนอกหน้าจอหรือถูกบีบ แต่หากตั้งค่าเป็น `wrap` วัตถุที่ล้นจะถูกตัดไปขึ้นบรรทัดใหม่โดยอัตโนมัติ เหมาะสำหรับการจัดโครงสร้างแท็กสไตล์ หรือเมนู Grid ภาพไอคอน

2. คุณสมบัติ gap (ความสะดวกใน SDK 54)

ในอดีตการเว้นช่องไฟระหว่างบล็อกสีจะต้องเขียน `marginRight` หรือ `marginBottom` ให้วุ่นวาย แต่ใน Expo SDK 54 เราสามารถใช้คำสั่งในการสร้างระยะห่างได้อย่างเป็นระเบียบ:

• `gap`: ใส่ค่าพิกเซลเว้นระยะทั้งแนวตั้งและแนวนอนเท่า ๆ กัน (เช่น `gap: 12`)

• `rowGap`: เว้นระยะห่างเฉพาะแถวแนวตั้ง

• `columnGap`: เว้นระยะห่างเฉพาะคอลัมน์แนวนอน

2.6 โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง

ด้านล่างนี้คือโค้ดตัวอย่างการจัดตำแหน่งแผงปุ่มเมนู 4 บล็อก (Grid Layout 2x2) โดยประยุกต์ใช้ Flexbox และช่องไฟ gap:

```
import { StyleSheet, Text, View } from 'react-native';

export default function GridLayout() {
```

```
  return (
    <View style={styles.container}>
      <Text style={styles.headerTitle}>เลย์เอาต์แบบตาราง 2x2</Text>

      <View style={styles.gridContainer}>
        {/* บล็อกที่ 1 */}
        <View style={[styles.box, { backgroundColor: '#ef4444' }]}>
          <Text style={styles.boxText}>เมนู A</Text>
        </View>
        {/* บล็อกที่ 2 */}
        <View style={[styles.box, { backgroundColor: '#3b82f6' }]}>
          <Text style={styles.boxText}>เมนู B</Text>
        </View>
        {/* บล็อกที่ 3 */}
        <View style={[styles.box, { backgroundColor: '#10b981' }]}>
          <Text style={styles.boxText}>เมนู C</Text>
        </View>
        {/* บล็อกที่ 4 */}
        <View style={[styles.box, { backgroundColor: '#f59e0b' }]}>
          <Text style={styles.boxText}>เมนู D</Text>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a',
    paddingTop: 60,
    paddingHorizontal: 20,
  },
  headerTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#ffffff',
    textAlign: 'center',
    marginBottom: 20,
  },
  gridContainer: {
    flexDirection: 'row',      // วางแนวนอน
    flexWrap: 'wrap',          // ถ้าล้นให้ตัดขึ้นบรรทัดใหม่
    gap: 16,                   // เว้นระยะขอบรอบด้านทุกกล่อง
    justifyContent: 'center', // กระจายชิดใน
  },
  box: {
    width: '45%',              // กินความกว้างเกือบครึ่งหนึ่ง
    height: 120,
    borderRadius: 16,
    justifyContent: 'center', // จัดแกนหลักกึ่งกลาง (แนวตั้ง)
```

```
    alignItems: 'center',      // จัดแกนรองกึ่งกลาง (แนวนอน)
  },
  boxText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
  },
});
```

สรุปท้ายบทเรียน

การเขียนและประกาศสไตล์ใน React Native: สไตล์ใน React Native ใช้แนวคิด CSS-in-JS ผ่านออบเจกต์ JavaScript โดยสามารถเขียนได้สองรูปแบบหลัก ได้แก่ Inline Styles ซึ่งเป็นการส่งออบเจกต์เข้าไปที่พร็อพส์สไตล์โดยตรง เหมาะสำหรับการทดสอบอย่างรวดเร็ว และ StyleSheet.create ซึ่งเป็นวิธีที่แนะนำเนื่องจากช่วยแยกส่วนสไตล์และทำให้ระบบสามารถประมวลผลพร้อมแคชสไตล์ได้อย่างมีประสิทธิภาพสูงขึ้น

โครงสร้างกล่องและคุณสมบัติขนาดขอบเขต: การควบคุมขนาดและพื้นที่ใน React Native อ้างอิงระบบ Box Model เช่นเดียวกับเว็บ ประกอบด้วยคุณสมบัติหลักอย่าง width และ height เพื่อกำหนดขนาดกล่อง, margin เพื่อระบุระยะห่างจากขอบกล่องภายนอกไปยังส่วนประกอบอื่น, padding สำหรับกำหนดช่องว่างระหว่างขอบด้านในกับเนื้อหาภายใน, border เพื่อระบุขนาดและสีขอบ รวมถึง borderRadius ที่ช่วยในการลบมุมกล่องให้มีความโค้งมน

ทิศทางการวางแกนด้วย flexDirection และ flex: การจัดตำแหน่งคอมโพเนนต์จะยึดหลักแนวแกน โดยคุณสมบัติ flexDirection จะกำหนดทิศทางการเรียงตัวของคอมโพเนนต์ลูก ซึ่งค่าเริ่มต้นใน React Native จะเป็นแบบแนวตั้ง (column) ต่างจากเว็บที่เป็นแนวนอน (row) เพื่อรองรับพฤติกรรมของแอปพลิเคชันมือถือ ในขณะที่คุณสมบัติ flex จะใช้สำหรับกำหนดสัดส่วนการยืดขยายและกระจายพื้นที่ว่างของแต่ละกล่อง

การจัดกึ่งกลางและการจัดเรียงด้วย justifyContent และ alignItems: การกระจายตัวของวัตถุลูกทำได้โดยใช้คุณสมบัติ justifyContent ในการจัดตำแหน่งตามแนวแกนหลัก (เช่น center, space-between หรือ space-around) และใช้ alignItems ในการจัดระเบียบวัตถุลูกตามแนวแกนรอง (เช่น stretch หรือ center) ซึ่งการผสมผสานสองคำสั่งนี้ช่วยให้จัดการเลย์เอาต์ที่ซับซ้อนและจัดข้อมูลให้อยู่กึ่งกลางหน้าจอได้อย่างเป็นระบบ

การขึ้นบรรทัดใหม่และการเว้นช่องว่าง: เมื่อวัตถุมีขนาดรวมเกินขอบเขตกล่องคอนเทนเนอร์ เราสามารถกำหนด flexWrap เป็น wrap เพื่อให้วัตถุขึ้นบรรทัดใหม่โดยอัตโนมัติ นอกจากนี้ใน Expo SDK 54 ยังรองรับการใช้งานคุณสมบัติ gap (Expo, 2024) rowGap และ columnGap ซึ่ง

ช่วยให้สามารถระบุช่องไฟหรือระยะห่างระหว่างบล็อกสีและวัตถุต่าง ๆ ได้สะดวกรวดเร็วโดยไม่ต้องประกาศ margin แยกรายตัว

โค้ดตัวอย่างหน้าเลย์เอาต์โครงสร้าง Grid จำลอง: ตัวอย่างการนำความรู้การจัดหน้าจอด้วย Flexbox มาใช้ร่วมกันจะเห็นได้ชัดจากการสร้างเลย์เอาต์แบบตาราง Grid ขนาด 2x2 โดยใช้ flexDirection: 'row' ร่วมกับ flexWrap: 'wrap' และการกำหนดระยะห่างด้วย gap โดยอาศัย alignItems: 'center' และ justifyContent: 'center' ในการจัดกล่องย่อยแต่ละใบให้เนื้อหาภายในอยู่ตรงกึ่งกลางอย่างสมบูรณ์

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับสไตล์ชีท (StyleSheet), กล่องสเกลขนาด (Box Model) และคุณสมบัติของระบบจัดตำแหน่งแบบ Flexbox ใน React Native Expo

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ใน React Native ข้อใดคือทิศทางการจัดเรียงแกนหลักเริ่มต้น (Default flexDirection) ของคอมโพเนนต์ลูกภายในกล่องคอนเทนเนอร์?

ก. `row`
ข. `column`
ค. `row-reverse`
ง. `column-reverse`

ข้อ 2: การระบุตัวเลขโดด ๆ ในการกำหนดขนาด เช่น `{ width: 150 }` ในสไตล์ของ React Native มีหน่วยวัดเป็นอะไร?

ก. Pixels (px)
ข. Percentage (%)
ค. Points / Density-independent Pixels (dp)
ง. Inches (in)

ข้อ 3: หากต้องการจัดเลย์เอาต์ให้กล่อง View ขนาดกว้าง 100 และสูง 100 มีหน้าตาเป็นวงกลมสมบูรณ์ จะต้องกำหนดคุณสมบัติ `borderRadius` ในสไตล์อย่างไร?

ก. `borderRadius: 100`
ข. `borderRadius: 50`
ค. `borderRadius: 25`

ง. `borderRadius: 10`

ข้อ 4: ข้อใดอธิบายความแตกต่างระหว่างคุณสมบัติ `margin` และ `padding` ได้อย่างถูกต้อง?

ก. `margin` คือขนาดเส้นขอบของวัตถุ ส่วน `padding` คือความโค้งมนของมุมกล่อง

ข. `margin` คือพื้นที่เว้นระยะรอบนอกขอบกล่อง ส่วน `padding` คือพื้นที่เว้นระยะรอบในขอบกล่องไปยังเนื้อหา

ค. `margin` มีผลกับทิศทางแนวตั้งเท่านั้น ส่วน `padding` มีผลเฉพาะทิศทางแนวนอน

ง. `margin` ใช้สำหรับข้อความ ส่วน `padding` ใช้สำหรับรูปภาพ

ข้อ 5: คุณสมบัติ `alignItems` ใน React Native ทำงานและควบคุมการจัดตำแหน่งในแนวแกนใด?

ก. ควบคุมการจัดเรียงในทิศทางแกนหลัก (Main Axis)

ข. ควบคุมการจัดเรียงในทิศทางแกนรอง (Cross Axis)

ค. ควบคุมขนาดของเส้นขอบกล่อง

ง. ควบคุมการหดตัวของขนาดตัวอักษร

ข้อ 6: หากเราเปิดกล่องคอนเทนเนอร์แนวนอนด้วย `flexDirection: 'row'` คุณสมบัติ `justifyContent` จะควบคุมการจัดตำแหน่งและจัดระเบียบข้อมูลตามทิศทางใด?

ก. แนวนอน (ซ้ายไปขวา)

ข. แนวตั้ง (บนลงล่าง)

ค. แนวทะแยงมุม

ง. หมุนตามเข็มนาฬิกา

ข้อ 7: ตัวเลือกข้อใดของ `justifyContent` ที่ช่วยกระจายกล่องตัวเลือกให้อยู่ห่างกันมากที่สุด โดยให้บล็อกแรกและบล็อกสุดท้ายชิดขอบข้างสุดของคอนเทนเนอร์พอดี?

ก. `space-around`

ข. `space-evenly`

ค. `space-between`

ง. `center`

ข้อ 8: เมื่อความกว้างรวมของบล็อกคอมโพเนนต์ลูกมีมากกว่าขนาดพื้นที่ของกล่องแม่ คุณสมบัติใดที่จะต้องตั้งค่าเพิ่มเติมเป็น `'wrap'` เพื่อจัดให้บล็อกที่ล้นหน้าจอตัดขึ้นแถวใหม่โดยอัตโนมัติ?

ก. `flexWrap`

ข. `flexDirection`

ค. `alignContent`

ง. `flexGrow`

ข้อ 9: ใน Expo SDK 54 คุณสมบัติข้อใดที่เพิ่มความสะดวกในการกำหนดระยะห่างระหว่างบล็อกวัตถุในคอนเทนเนอร์แนวตั้งและแนวนอนพร้อมกันโดยไม่ต้องเขียนระยะขอบแยกทีละชิ้น?

ก. `marginHorizontal`

ข. `gap`

ค. `spacing`

ง. `paddings`

ข้อ 10: หากมีบล็อกลูกสองกล่องอยู่ในคอนเทนเนอร์เดียวกัน และกล่องแรกถูกกำหนดให้มีสไตล์ `flex: 1` ในขณะที่กล่องที่สองกำหนดเป็น `flex: 2` ข้อใดอธิบายการจัดสรรสัดส่วนพื้นที่ว่างได้ถูกต้อง?

ก. กล่องแรกจะได้ความยาว 100px และกล่องที่สองได้ 200px

ข. กล่องแรกและกล่องที่สองจะกินพื้นที่เท่ากันโดยสลับกัน

ค. ระบบจะแบ่งพื้นที่เป็น 3 ส่วนเท่า ๆ กัน โดยกล่องแรกจะได้ 1 ส่วน และกล่องที่สองจะได้ 2 ส่วนที่เหลือ

ง. กล่องแรกจะซ้อนอยู่บนกล่องที่สองเนื่องจากค่าดัชนีน้อยกว่า

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `row` เป็นค่าเริ่มต้นสำหรับ CSS บนบราวเซอร์ทั่วไป แต่ไม่ใช่ของ React Native

• ข. ถูก: React Native ออกแบบมาสำหรับมือถือซึ่งเน้นการอ่านและการเลื่อนลงแนวตั้งเป็นหลัก ดังนั้นทิศทางแกนหลักเริ่มต้น (`flexDirection`) จึงถูกกำหนดเป็น `column` (แนวตั้ง) โดยอัตโนมัติ

• ค. และ ง. ผิด: เป็นรูปแบบการสลับย้อนทางแกน (Reverse) ซึ่งไม่ใช่ค่ามาตรฐานเริ่มต้น

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• **ก. ผิด:** React Native ไม่ได้กำหนดสไตล์ด้วยพิกเซลกายภาพตรงตัวเพราะขนาดจอและปริมาณพิกเซลแต่ละเครื่องต่างกันมาก

• **ข. ผิด:** เปอร์เซ็นต์จะทำงานเฉพาะเมื่อใส่ค่าเป็นข้อความสตริง เช่น `'50%'` เท่านั้น

• **ค. ถูก:** ค่าตัวเลขในสไตล์ชีทของ React Native คือหน่วยจุดวัดเชิงพื้นที่ที่ไม่ขึ้นกับความหนาแน่นพิกเซลของจอแต่ละรุ่น (เรียกว่า Points ใน iOS หรือ Density-independent Pixels - dp ใน Android) ซึ่งจะสเกลพิกเซลจริงขึ้นตามหน้าจออุปกรณ์ปลายทางให้อัตโนมัติ

• **ง. ผิด:** นิ้วไม่ใช่หน่วยวัดของเลย์เอาต์หน้าจอแอปพลิเคชัน

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การตั้งค่าเท่าความกว้างจะทำให้โค้งมนล้นหรืออาจกลายเป็นรูปแคปซูลหากสัดส่วนอื่นเบี้ยว

• **ข. ถูก:** ตามสูตรเลขคณิตการตัดมุมกล่องสี่เหลี่ยมจัตุรัสให้กลายเป็นวงกลมสมบูรณ์ขอบโค้งมน (`borderRadius`) จะต้องมีขนาดเท่ากับ **ครึ่งหนึ่ง** ของความยาวกว้างและสูง (กว้าง 100, สูง 100 -> ครึ่งหนึ่งคือ 50)

• **ค. และ ง. ผิด:** จะได้มุมกล่องโค้งงอเฉย ๆ แต่ขอบยังเป็นทรงสี่เหลี่ยมด้านเท่า

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ขอบวัตถุและมุมโค้งใช้อีกคลาสคือ `borderWidth` และ `borderRadius`

• **ข. ถูก:** ในสถาปัตยกรรม Box Model ของการวางเลย์เอาต์ `margin` (ระยะขอบภายนอก) จะช่วยสร้างช่องไฟว่าง ๆ ผลักตัวบล็อกให้ออกห่างจากกล่องอื่น ส่วน `padding` (ระยะขอบภายใน) จะใช้เว้นช่องไฟภายในกล่องสีของตนเองเพื่อไม่ให้ตัวหนังสือชิดขอบในเกินไป

• **ค. และ ง. ผิด:** ทั้งสองคำสั่งทำงานได้ทุกแนวทิศทางและควบคุมช่องว่างไม่ได้จำกัดประเภทเนื้อหา

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** แกนหลักจะถูกควบคุมและเรียงบล็อกผ่านคุณสมบัติ `justifyContent`

• **ข. ถูก:** `alignItems` จะทำหน้าที่ควบคุมและจัดแนวระเบียบให้กับคอมโพเนนต์ย่อยตามทิศทางแนวแกนรอง (Cross Axis) เสมอ (เช่น หากแถวเรียงตัวแนวนอน แกนรองจะเป็นแนวตั้ง)

• **ค. และ ง. ผิด:** ควบคุมขนาดเส้นขอบและตัวอักษรจะใช้ `borderWidth` และ `fontSize`

เฉลย ข้อ 6: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: หากมีการกำหนด `flexDirection: 'row'` (เรียงแนวนอน) ทิศทางแกนหลัก (Main Axis) จะสลับมาอยู่ทางแนวนอนทันที ส่งผลให้ `justifyContent` ทำหน้าที่ผลักหรือจัดบล็อกเรียงชิดตามแนวนอน (ซ้าย-ขวา) ไปด้วย

• ข. ผิด: แกนแนวตั้งจะกลายเป็นแกนรอง (Cross Axis) และถูกควบคุมโดย `alignItems` แทน

• ค. และ ง. ผิด: ระบบ Flexbox ไม่มีการจัดมุมหรือหมุนทิศทางทะแยงในการทำงานหลัก

เฉลย ข้อ 7: ตอบ ค.

• เหตุผลประกอบ:

• ก. ผิด: `space-around` จะแบ่งช่องว่างระหว่างบล็อกเท่า ๆ กันและเหลือช่องว่างข้างหน้า/ท้ายครึ่งหนึ่ง

• ข. ผิด: `space-evenly` จะแบ่งช่องว่างให้กว้างเท่ากันทั้งหมดอย่างเท่าเทียมรวมถึงขอบริมด้วย

• ค. ถูก: `space-between` จะกระจายบล็อกลูกชิดซ้ายและชิดขวาสุดริมขอบหน้าจอพอดี และกระจายสัดส่วนสเปซช่องว่างที่เหลือไว้คั่นตรงกึ่งกลางระว่างกล่องทั้งหมดเท่ากัน

• ง. ผิด: `center` จะรวมกล่องทั้งหมดชิดกึ่งกลางแกน

เฉลย ข้อ 8: ตอบ ก.

• เหตุผลประกอบ:

• ก. ถูก: คุณสมบัติ `flexWrap: 'wrap'` จะคอยสั่งการตัว Bunder เลย์เอาต์ว่า หากความกว้างสะสมของคอมโพเนนต์ลูกเกินพื้นที่คอนเทนเนอร์แม่ ให้ผลักบล็อกถัดไปลงมาขึ้นแถวใหม่ด้านล่างทันที

• ข. ผิด: `flexDirection` ใช้กำหนดเพียงทิศทางการไหลหลักของแกนว่าวางตัวแนวตั้งหรือนอน

• ค. และ ง. ผิด: ใช้สำหรับคำสั่งสัดส่วนการหดและจัดแนวย่อยเมื่อมีหลายแถว

เฉลย ข้อ 9: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: `marginHorizontal` จะเว้นขอบริมภายนอกทั้งซ้ายและขวาของกล่องแต่ละตัว แต่ไม่มีผลกึ่งกลางระหว่างบล็อกย่อย

• **ข. ถูก:** คำสั่ง `gap` (มีพร้อมใช้งานใน SDK ใหม่) ช่วยให้นักพัฒนาสร้างช่องไฟเว้นระหว่างบล็อกย่อยภายในคอนเทนเนอร์ Flexbox ได้ง่ายที่สุด โดยจะคั่นระหว่างบล็อกเท่านั้น โดยไม่ต้องคอยระบุหักลบขอบริมด้านซ้ายขวาของแต่ละกล่อง

• **ค. และ ง. ผิด:** ไม่ใช่ชื่อคุณสมบัติมาตรฐานที่จัดทำในระบบสไตล์ React Native

เฉลย ข้อ 10: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ค่าตัวเลขใน `flex` ไม่ได้หมายความถึงพิกเซล (px) แบบตายตัว แต่เป็นหน่วยอัตราส่วน

• **ข. ผิด:** จะกินสัดส่วนขนาดที่ไม่เท่ากันเพราะตัวเลขต่างกัน

• **ค. ถูก:** คุณสมบัติ `flex` ทำงานโดยหาผลรวมค่าสะสมทั้งหมด (ในโจทย์ 1 + 2 = 3 ส่วน) แล้วแบ่งสเกลสัดส่วนพื้นที่ว่างออกเป็นจำนวนส่วนรวมนั้น จากนั้นจึงนำไปปันสัดส่วนให้กับกล่องตามค่าที่ระบุ (กล่องแรกได้ 1/3 ส่วน และกล่องที่สองได้ 2/3 ส่วน)

• **ง. ผิด:** การซ้อนทับภาพขึ้นอยู่กับลำดับการโค้ดหรือคุณสมบัติ `zIndex` ไม่ใช่ `flex`

ปฏิบัติการท้ายบทเรียน: การจัดเลย์เอาต์หน้าจอแดชบอร์ดจำลองด้วย Flexbox

ปฏิบัติการนี้จะฝึกทักษะการประยุกต์ใช้ระบบ Flexbox ในการออกแบบโครงสร้างส่วนจัดตำแหน่งต่าง ๆ บนหน้าจอจริง โดยจำลองหน้าอินเทอร์เฟซของ **แอปพลิเคชันแดชบอร์ดควบคุมสมาร์ทโฮม (Smart Home Dashboard Mockup)** ผ่านคอมโพเนนต์พื้นฐาน `View` และ `Text`

1. วัตถุประสงค์การเรียนรู้

1. สามารถใช้คุณสมบัติ `flexDirection` ในการวางวัตถุทั้งในแนวแกนตั้งและแนวนอนได้ตามโจทย์กำหนด

2. สามารถใช้คุณสมบัติ `justifyContent` และ `alignItems` ในการกระจายบล็อกสีและจัดวัตถุกึ่งกลางหน้าจอได้อย่างถูกต้อง

3. สามารถใช้งานคุณสมบัติ `flexWrap` และ `gap` เพื่อสร้างรูปแบบ Grid ตารางเมนู 2x2 ได้สำเร็จ

4. มีความเข้าใจหลักการจัดลำดับการเขียนและประกอบสไตล์ชีทภายนอกด้วย `StyleSheet.create`

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนจะต้องเขียนโค้ดเพื่อสร้างหน้าจอแดชบอร์ดจำลองในไฟล์ `app/index.tsx` (หลังจากรัน `npm run reset-project` จากบทที่ 1 แล้ว) โดยมีรายละเอียดเลย์เอาต์ 4 ส่วนย่อยหลัก ดังนี้:

```
+-----------------------------------------+
|  [Header Bar] (Title & Notification Dot) |
+-----------------------------------------+
|  [Status Summary Card] (Full Width)      |
+-----------------------------------------+
|  [2x2 Menu Grid Block]                   |
|  - [Menu 1 (47%)]    - [Menu 2 (47%)]    |
|  - [Menu 3 (47%)]    - [Menu 4 (47%)]    |
+-----------------------------------------+
|  [Footer Navigation Bar]                 |
+-----------------------------------------+
```

รายละเอียดเงื่อนไขการจัดสไตล์:

1. **ธีมพื้นหลังแอปพลิเคชัน:** กำหนดเป็นสีเทาดำหรูหรา (`#0f172a`)
2. **ส่วนหัว (Header Bar):**
   - จัดเรียงแบบแนวนอน (`flexDirection: 'row'`)
   - แบ่งตัวหนังสือชิดซ้ายและขวาสุดริมขอบ (`justifyContent: 'space-between'`)
   - มีชื่อแอป "HOME CONTROL" และฝั่งขวาแสดงจุดไอคอนกลมสีเขียวนีออนแสดงสถานะ Online
3. **การ์ดสรุปสถานะ (Status Summary Card):**
   - กำหนดความกว้างเต็มจอ และมีมุมโค้งมน
   - กำหนดสีพื้นหลังการ์ดเป็นสี Slate กึ่งโปร่งใส (`#1e293b`)
   - ภายในประกอบด้วยข้อความบอกค่าอุณหภูมิ เช่น "24°C - ปกติ"
4. **ตารางบล็อกเมนูย่อย (2x2 Grid):**
   - บล็อกปุ่มเมนูทั้ง 4 ปุ่มมีขนาดความกว้างบล็อกละ **47%** ของพื้นที่ เพื่อให้เรียงตัวคู่กันสองข้างได้พอดี
   - จัดการล้นของวัตถุด้วย `flexWrap: 'wrap'` และมีช่องไฟเว้นระหว่างบล็อกสีเท่ากับ **16** พิกเซล
   - แต่ละบล็อกสีมีพื้นหลังคนละสีกันเพื่อแสดงความเป็นเมนูเฉพาะด้าน
5. **แถบส่วนท้าย (Footer Navigation Bar):**

• ลอยตัวชิดด้านล่างสุดของหน้าจอ

• แสดงแท็บจำลอง 3 เมนู (เช่น "Home", "Devices", "Settings") เรียงแนวนอนกระจายระยะห่างสเปซเท่ากันทั้งหมด (`justifyContent: 'space-evenly'`)

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: เตรียมโปรเจคเริ่มต้น

ตรวจสอบความพร้อมของไฟล์ `app/index.tsx` หากมีโค้ดเก่าอยู่ให้ลบออก หรือเปิดเซิร์ฟเวอร์ด้วยคำสั่ง `npx expo start` เพื่อสแตนด์บายทดสอบผลลัพธ์ผ่านหน้าจออุปกรณ์จริงของคุณ

ขั้นตอนที่ 2: เขียนโค้ดแบ่งโครงสร้างหลัก (Layout Partitioning)

คัดลอกตัวอย่างโครงสร้างโค้ดเริ่มต้นนี้ลงไปใน `app/index.tsx` เพื่อแบ่งโครงสร้างหลักทั้ง 4 ส่วน และทดลองวิเคราะห์ว่าแกนหลักแกนรองทำงานอย่างไร:

```
import { StyleSheet, Text, View } from 'react-native';

export default function SmartHomeDashboard() {
  return (
    <View style={styles.container}>

      {/* 1. Header Bar */}
      <View style={styles.header}>
        <Text style={styles.headerText}>HOME CONTROL</Text>
        <View style={styles.statusContainer}>
          <View style={styles.statusDot} />
          <Text style={styles.statusDotText}>ONLINE</Text>
        </View>
      </View>

      {/* 2. Status Summary Card */}
      <View style={styles.statusCard}>
        <Text style={styles.cardTitle}>อุณหภูมิห้องโดยรวม</Text>
        <Text style={styles.cardValue}>24°C</Text>
        <Text style={styles.cardStatus}>ระบบปรับอากาศกำลังทำงานระดับปกติ</Text>
      </View>

      {/* 3. 2x2 Menu Grid Block */}
      <View style={styles.menuGrid}>

        {/* บล็อกเมนู 1 */}
        <View style={[styles.menuItem, { backgroundColor: '#ea580c' }]}>
          <Text style={styles.menuEmoji}>💡</Text>
          <Text style={styles.menuText}>ห้องนั่งเล่น</Text>
        </View>

        {/* บล็อกเมนู 2 */}
```

```
          <View style={[styles.menuItem, { backgroundColor: '#2563eb' }]}>
            <Text style={styles.menuEmoji}>🔒</Text>
            <Text style={styles.menuText}>ความปลอดภัย</Text>
          </View>

          {/* บล็อกเมนู 3 */}
          <View style={[styles.menuItem, { backgroundColor: '#16a34a' }]}>
            <Text style={styles.menuEmoji}>🔌</Text>
            <Text style={styles.menuText}>เครื่องใช้ไฟฟ้า</Text>
          </View>

          {/* บล็อกเมนู 4 */}
          <View style={[styles.menuItem, { backgroundColor: '#db2777' }]}>
            <Text style={styles.menuEmoji}>🎵</Text>
            <Text style={styles.menuText}>เครื่องเสียง</Text>
          </View>

        </View>

        {/* 4. Footer Navigation Bar */}
        <View style={styles.footer}>
          <Text style={styles.footerItemActive}>หน้าหลัก</Text>
          <Text style={styles.footerItem}>อุปกรณ์</Text>
          <Text style={styles.footerItem}>ตั้งค่า</Text>
        </View>

      </View>
    );
}

const styles = StyleSheet.create({
  // ให้ผู้เรียนระบุสไตล์ชีทสำหรับจัดแนวทาง Flexbox ที่ขั้นตอนถัดไป
});
```

ขั้นตอนที่ 3: ระบุสไตล์และคลาสการจัดตำแหน่งด้วย Flexbox

เขียนข้อกำหนดสไตล์ลงในฟังก์ชัน `StyleSheet.create` ด้านล่างสุดของไฟล์ `app/index.tsx` ดังนี้ เพื่อจัดตำแหน่งกล่องตามโจทย์กำหนด:

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // ธีมพื้นหลัง Dark Mode Slate
    paddingTop: 60,             // เว้นหลบหน้าจอด้านบน
    paddingHorizontal: 20,
    justifyContent: 'space-between', // ช่วยดึงส่วนหัว ตัวเนื้อหา และส่วนท้ายแยกชิดขอบห่างจากกัน
    paddingBottom: 20,
  },
```

```
  header: {
    flexDirection: 'row',          // เรียงแนวนอน
    justifyContent: 'space-between', // ดึงหัวหนังสือชิดซ้าย จุดเขียวชิดขวา
    alignItems: 'center',          // จัดวัตถุกึ่งกลางแนวแกนรอง (แนวตั้ง)
    borderBottomWidth: 1,
    borderColor: '#1e293b',
    paddingBottom: 12,
  },
  headerText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    letterSpacing: 1,
  },
  statusContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
  },
  statusDot: {
    width: 8,
    height: 8,
    borderRadius: 4,               // ทำสัญลักษณ์วงกลมสมบูรณ์
    backgroundColor: '#22c55e',    // สีเขียวนีออน
  },
  statusDotText: {
    color: '#22c55e',
    fontSize: 12,
    fontWeight: 'bold',
  },
  statusCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    width: '100%',
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 10,
  },
  cardTitle: {
    color: '#94a3b8',
    fontSize: 14,
    marginBottom: 4,
  },
  cardValue: {
    color: '#ffffff',
    fontSize: 32,
    fontWeight: 'bold',
    marginBottom: 4,
  },
  cardStatus: {
    color: '#38bdf8',
    fontSize: 12,
  },
```

```
  menuGrid: {
    flexDirection: 'row',          // จัดวัตถุลูกวางแนวนอน
    flexWrap: 'wrap',              // ถ้าวัตถุเกินความกว้าง ให้ตัดขึ้นแถวใหม่
    gap: 16,                       // เว้นระยะขอบรอบทุกด้าน
    justifyContent: 'space-between', // ผลักสัดส่วนเมนูสองฝั่งให้ออกชิดขอบ
    marginVertical: 10,
  },
  menuItem: {
    width: '47%',                  // ความกว้างกล่องต่อชิ้น (แบ่งคู่ได้ 2 บล็อกพอดีและเหลือที่เว้น
gap)
    height: 110,
    borderRadius: 16,
    padding: 16,
    justifyContent: 'space-between', // ช่วยดึง Emoji อยู่บน ชื่อข้อความอยู่ข้างล่าง
  },
  menuEmoji: {
    fontSize: 28,
  },
  menuText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 16,
  },
  footer: {
    flexDirection: 'row',
    justifyContent: 'space-evenly', // กระจายพื้นที่ว่างแบ่งเท่า ๆ กันระหว่าง 3 ปุ่มเมนู
    backgroundColor: '#1e293b',
    paddingVertical: 14,
    borderRadius: 24,
    borderWidth: 1,
    borderColor: '#334155',
  },
  footerItem: {
    color: '#64748b',
    fontWeight: 'bold',
  },
  footerItemActive: {
    color: '#38bdf8',              // ปรับสีเมนูแรกเป็นสีฟ้านีออนแสดงการแอ็คทีฟหน้าปัจจุบัน
    fontWeight: 'bold',
  },
});
```

ขั้นตอนที่ 4: ทดสอบการทำงานและปรับเปลี่ยนพฤติกรรมเลย์เอาต์ (Experiment)

ให้ผู้เรียนบันทึกไฟล์และสังเกตแดชบอร์ดจำลองบนสมาร์ทโฟน จากนั้นให้ทดลองทดสอบพฤติกรรมสไตล์ดังนี้:

1. ทดลองเปลี่ยนค่า `flexDirection: 'row'` ในสไตล์ `menuGrid` ให้เป็น `'column'` สังเกตว่าเมนู 4 บล็อกถูกจัดเรียงอย่างไร

2. ลองแก้ไขค่า gap: 16 ในเมนูตารางให้ลดลงเหลือ 8 หรือเปลี่ยนเป็น rowGap และ columnGap เพื่อเข้าใจความยืดหยุ่นของตัวแปร

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกโค้ดโครงสร้างที่รวมคอมโพเนนต์และสไตล์ชีทเข้าด้วยกันฉบับสมบูรณ์ด้านล่างนี้ ไปใส่แทนที่ในไฟล์ app/index.tsx เพื่อตรวจสอบการแสดงผลหน้าจอแดชบอร์ดควบคุมสมาร์ทโฮมที่ถูกต้อง:

```
import { StyleSheet, Text, View } from 'react-native';

export default function SmartHomeDashboard() {
  return (
    <View style={styles.container}>

      {/* 1. Header Bar */}
      <View style={styles.header}>
        <Text style={styles.headerText}>HOME CONTROL</Text>
        <View style={styles.statusContainer}>
          <View style={styles.statusDot} />
          <Text style={styles.statusDotText}>ONLINE</Text>
        </View>
      </View>

      {/* 2. Status Summary Card */}
      <View style={styles.statusCard}>
        <Text style={styles.cardTitle}>อุณหภูมิห้องโดยรวม</Text>
        <Text style={styles.cardValue}>24°C</Text>
        <Text style={styles.cardStatus}>ระบบปรับอากาศกำลังทำงานระดับปกติ</Text>
      </View>

      {/* 3. 2x2 Menu Grid Block */}
      <View style={styles.menuGrid}>

        {/* บล็อกเมนู 1 */}
        <View style={[styles.menuItem, { backgroundColor: '#ea580c' }]}>
          <Text style={styles.menuEmoji}>💡</Text>
          <Text style={styles.menuText}>ห้องนั่งเล่น</Text>
        </View>

        {/* บล็อกเมนู 2 */}
        <View style={[styles.menuItem, { backgroundColor: '#2563eb' }]}>
          <Text style={styles.menuEmoji}>🔒</Text>
          <Text style={styles.menuText}>ความปลอดภัย</Text>
        </View>

        {/* บล็อกเมนู 3 */}
        <View style={[styles.menuItem, { backgroundColor: '#16a34a' }]}>
          <Text style={styles.menuEmoji}>🔌</Text>
```

```
          <Text style={styles.menuText}>เครื่องใช้ไฟฟ้า</Text>
        </View>

        {/* บล็อกเมนู 4 */}
        <View style={[styles.menuItem, { backgroundColor: '#db2777' }]}>
          <Text style={styles.menuEmoji}>🎵</Text>
          <Text style={styles.menuText}>เครื่องเสียง</Text>
        </View>

      </View>

      {/* 4. Footer Navigation Bar */}
      <View style={styles.footer}>
        <Text style={styles.footerItemActive}>หน้าหลัก</Text>
        <Text style={styles.footerItem}>อุปกรณ์</Text>
        <Text style={styles.footerItem}>ตั้งค่า</Text>
      </View>

    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#0f172a', // ธีมพื้นหลัง Dark Mode Slate
    paddingTop: 60,             // เว้นหลบหน้าจอด้านบน
    paddingHorizontal: 20,
    justifyContent: 'space-between', // ช่วยดึงส่วนหัว ตัวเนื้อหา และส่วนท้ายแยกชิดขอบห่างจากกัน
    paddingBottom: 20,
  },
  header: {
    flexDirection: 'row',         // เรียงแนวนอน
    justifyContent: 'space-between', // ดึงหัวหนังสือชิดซ้าย จุดเขียวชิดขวา
    alignItems: 'center',         // จัดวัตถุกึ่งกลางแนวแกนรอง (แนวตั้ง)
    borderBottomWidth: 1,
    borderColor: '#1e293b',
    paddingBottom: 12,
  },
  headerText: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    letterSpacing: 1,
  },
  statusContainer: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
  },
```

```
  statusDot: {
    width: 8,
    height: 8,
    borderRadius: 4,              // ทำสัญลักษณ์วงกลมสมบูรณ์
    backgroundColor: '#22c55e',   // สีเขียวนีออน
  },
  statusDotText: {
    color: '#22c55e',
    fontSize: 12,
    fontWeight: 'bold',
  },
  statusCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    width: '100%',
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 10,
  },
  cardTitle: {
    color: '#94a3b8',
    fontSize: 14,
    marginBottom: 4,
  },
  cardValue: {
    color: '#ffffff',
    fontSize: 32,
    fontWeight: 'bold',
    marginBottom: 4,
  },
  cardStatus: {
    color: '#38bdf8',
    fontSize: 12,
  },
  menuGrid: {
    flexDirection: 'row',         // จัดวัตถุลูกวางแนวนอน
    flexWrap: 'wrap',             // ถ้าวัตถุเกินความกว้าง ให้ตัดขึ้นแถวใหม่
    gap: 16,                      // เว้นระยะขอบรอบทุกด้าน
    justifyContent: 'space-between', // ผลักสัดส่วนเมนูสองฝั่งให้ออกชิดขอบ
    marginVertical: 10,
  },
  menuItem: {
    width: '47%',                 // ความกว้างกล่องต่อชิ้น (แบ่งคู่ได้ 2 บล็อกพอดีและเหลือที่เว้น
gap)
    height: 110,
    borderRadius: 16,
    padding: 16,
    justifyContent: 'space-between', // ช่วยดึง Emoji อยู่บน ชื่อข้อความอยู่ข้างล่าง
  },
  menuEmoji: {
    fontSize: 28,
  },
```

```
  menuText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 16,
  },
  footer: {
    flexDirection: 'row',
    justifyContent: 'space-evenly', // กระจายพื้นที่ว่างแบ่งเท่า ๆ กันระหว่าง 3 ปุ่มเมนู
    backgroundColor: '#1e293b',
    paddingVertical: 14,
    borderRadius: 24,
    borderWidth: 1,
    borderColor: '#334155',
  },
  footerItem: {
    color: '#64748b',
    fontWeight: 'bold',
  },
  footerItemActive: {
    color: '#38bdf8',              // ปรับสีเมนูแรกเป็นสีฟ้านีออนแสดงการแอ็คทีฟหน้าปัจจุบัน
    fontWeight: 'bold',
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *User Interface and Styling*. จาก https://docs.expo.dev/versions/v54.0.0/develop/user-interface/overview/

React Native. (2024). *Height and Width*. จาก https://reactnative.dev/docs/height-and-width

React Native. (2024). *Layout with Flexbox*. จาก https://reactnative.dev/docs/flexbox